NLP 243 Deep Learning for Natural Language Processing

Fall 2023

Gradient Descent, Activation and Loss Functions

Outline for Today

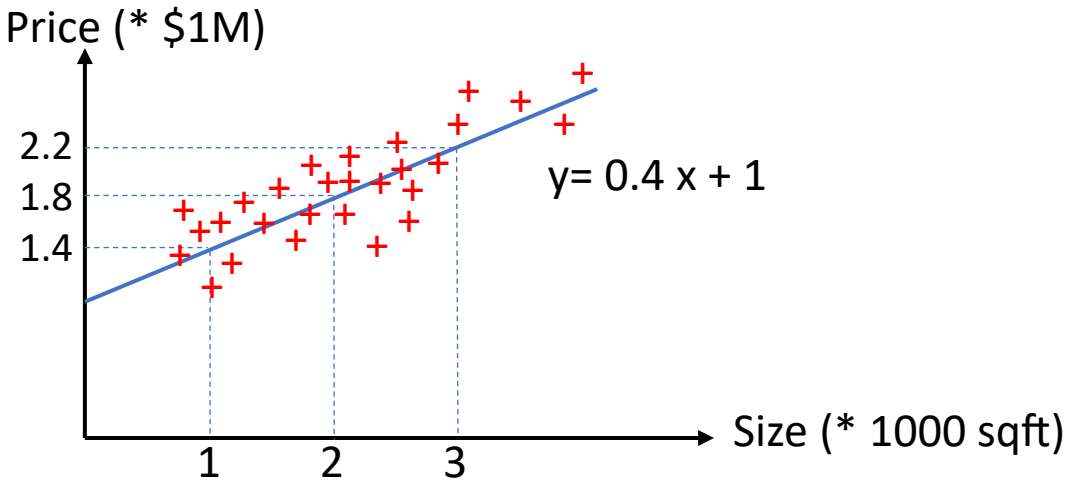
- Linear Regression
- Gradient Descent (and versions)
- Practical Tips
- Softmax Regression
- Activation and Loss Functions using PyTorch

Linear Regression (cont.)

- **Regression**: modeling the relationship between data points **x** and corresponding real-valued targets y
  - Example: Predicting prices of homes
  - Label: Price, $y^{(i)}$
  - Features: Size, Age
    - $x^{(i)}=[x^{(i)}_1, x^{(i)}_2]$

2019 Sales:

Size (sqft)	Price
2100	$1.90M
1600	$1.56M
3200	$2.40M
…	…



Linear Regression (cont.)

- Assumption:
  - the relationship between the *features* $\mathbf{x}$ and targets y is linear,
  - i.e., y can be expressed as a weighted sum of the inputs $\mathbf{x}$, give or take some noise on the observations

$$\text{price} = w_1 . x_1 + w_2 . x_1 + b$$

weights

Bias
(offset or intercept)

Linear Regression (cont.)

- Goal: choose the weights w and bias b to best fit the true values of y observed in the data
- When our inputs consist of d features, we express our prediction $\hat{y}$ as:

$$\hat{y} = w_1 \cdot x_1 + \ldots + w_d \cdot x_d + b$$

- Collecting all features into a vector $\mathbf{x}$ and all weights into a vector $\mathbf{w}$, our model can be expressed compactly using a dot product:

$$\hat{y} = \mathbf{w}^T \mathbf{x} + b$$

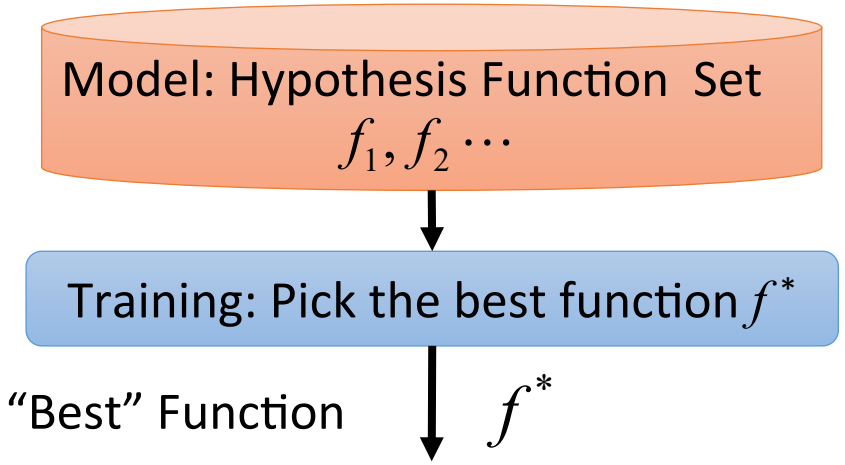
- $\mathbf{x}$**:** feature vector for a single data point
- $\mathbf{X}$**:** a collection of data points
- $\hat{\mathbf{y}}$: vector of estimated predictions

$$\hat{\mathbf{y}} = \mathbf{X}\mathbf{w} + b$$

Training Procedure: Searching for the best parameters

$$\hat{\mathbf{y}} = \mathbf{X}\mathbf{w} + b$$

Parameters: elements of $\mathbf{w}$ **and** b



- Q1. What is the model? (function hypothesis set)
- Q2. What does a "good" function mean?
- Q3. How do we find the "best" function?

Loss Function

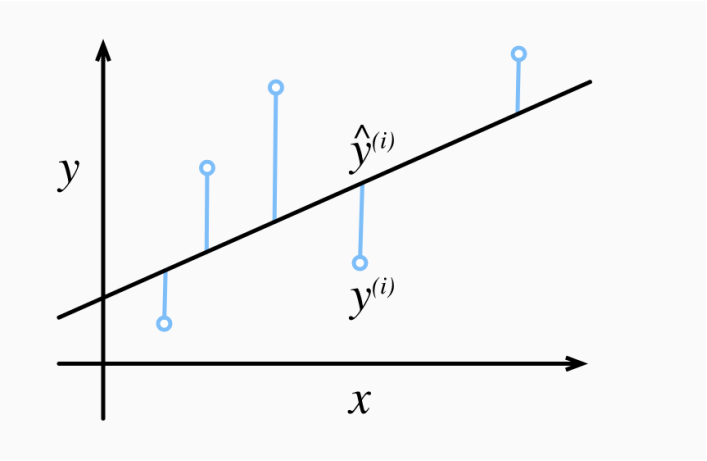
- Measure of *fitness*, quantifies the distance between $\mathbf{y}$ **and** $\hat{\mathbf{y}}$.
- Sum of squared errors:

$$l^{(i)}(\mathbf{w},b) = \frac{1}{2}(\hat{y}^{(i)} - y^{(i)})^2$$

- To measure quality over the entire training set, we average over all K training examples:

$$L(\mathbf{w},b) = \frac{1}{K}\sum_{i=1}^{K} l^{(i)}(\mathbf{w},b)$$

$$= \frac{1}{K}\sum_{i=1}^{K} \frac{1}{2}\left(\mathbf{w}^\top \mathbf{x}^{(i)} + b - y^{(i)}\right)^2$$



Residual = truth - predicted

Training Procedure (cont.)

- Training the model ≈ search for parameters ($\mathbf{w}^*, b^*$) that minimize the total loss across all training samples:

$$\mathbf{w}^*, b^* = \text{argmin}_{\mathbf{w},b}\, L(\mathbf{w}, b)$$

Simplification in Notation

- $\hat{\mathbf{y}} = \mathbf{X}\mathbf{w} + b$
- Append 1 to features and bias to the weights: **X -> [X 1] and w ->** $\begin{bmatrix} \mathbf{w} \\ b \end{bmatrix}$

$$\begin{pmatrix} x_1^{(1)} & x_2^{(1)} & \dots & x_n^{(1)} \\ x_1^{(2)} & x_2^{(2)} & \dots & x_n^{(2)} \\ \dots & & & \\ x_1^{(K)} & x_2^{(K)} & \dots & x_n^{(K)} \end{pmatrix} \begin{pmatrix} w_1 \\ w_2 \\ \dots \\ w_n \end{pmatrix} + b = \begin{pmatrix} x_1^{(1)} & x_2^{(1)} & \dots & x_n^{(1)} & 1 \\ x_1^{(2)} & x_2^{(2)} & \dots & x_n^{(2)} & 1 \\ \dots & & & & \\ x_1^{(K)} & x_2^{(K)} & \dots & x_n^{(K)} & 1 \end{pmatrix} \begin{pmatrix} w_1 \\ w_2 \\ \dots \\ w_n \\ b \end{pmatrix} = \mathbf{X}\,\mathbf{w}$$

Solution for Linear Regression

$$L(\mathbf{X}, y, \mathbf{w}) = \frac{1}{K} \, || \, y - \mathbf{X}.\mathbf{w} \, ||^{2}$$

$$\frac{d}{d\mathbf{w}} L(\mathbf{X}, y, \mathbf{w}) = \frac{2}{K} (y - \mathbf{X}.\mathbf{w})^{\mathsf{T}} \mathbf{X}$$

Loss is convex, hence the optimum solution is at: $\frac{d}{d\mathbf{w}} L(\mathbf{X}, y, \mathbf{w}) = 0$

Therefore,

$$\frac{2}{K} (y - \mathbf{X}.\mathbf{w}^*)^{\mathsf{T}} \mathbf{X} = 0 \qquad \text{and} \qquad \mathbf{w}^* = (\mathbf{X}^{\mathsf{T}} \mathbf{X})^{-1} \mathbf{X} \, y$$

Outline for Today

- Linear Regression
- Gradient Descent (and versions)
- Practical Tips
- Softmax Regression
- Activation and Loss Functions using PyTorch

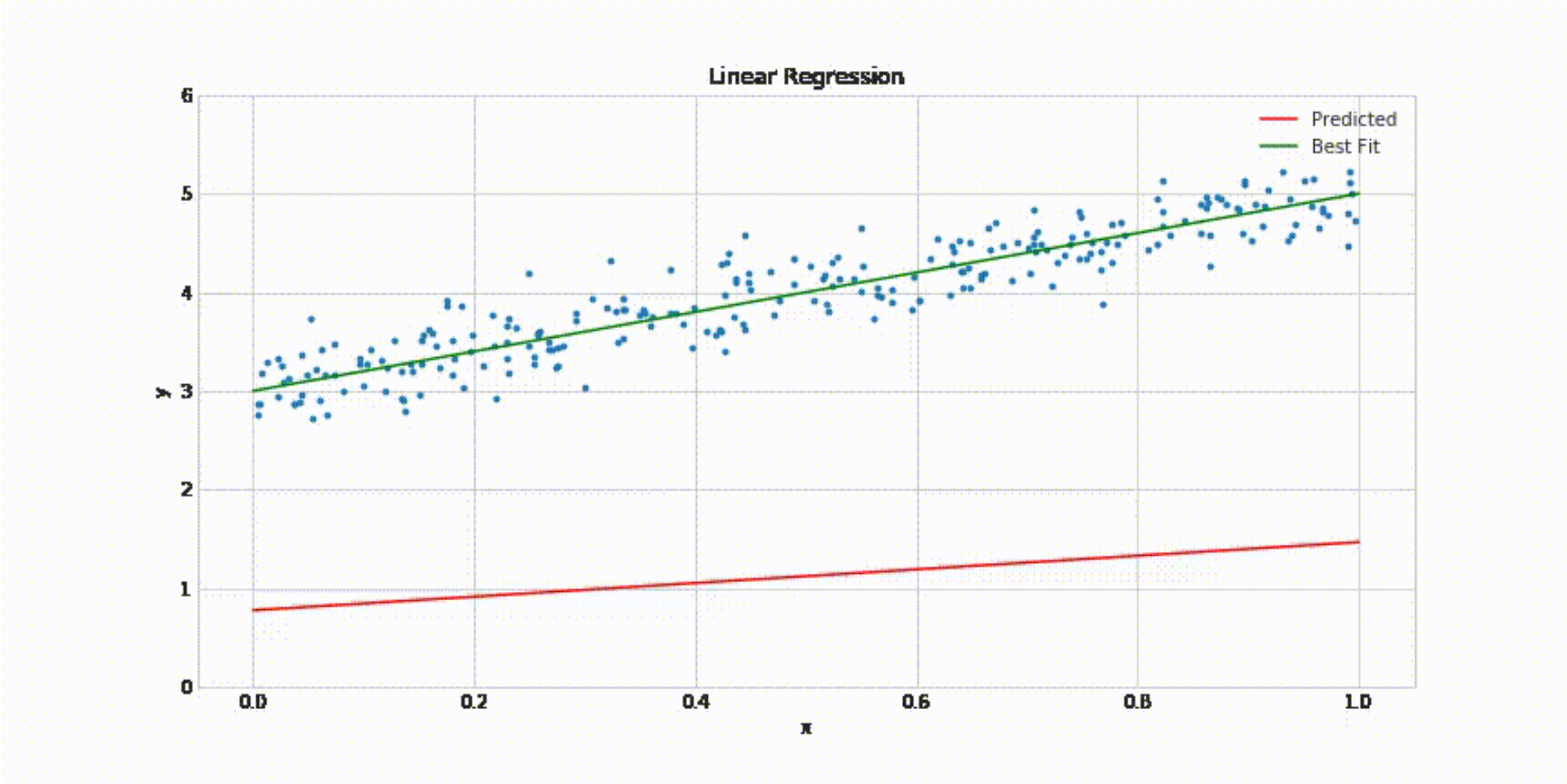
Gradient Descent

- Even when we cannot solve the models analytically, we can still train models effectively by **iteratively reducing the error by updating the parameters in the direction that incrementally lowers the loss function:**

  - Derivative of the true loss (i.e., average of the losses computed over all examples in the training data).
  - Expressing the updates mathematically:

$$(\mathbf{w},b) \leftarrow (\mathbf{w},b) - \frac{\eta}{|K|} \sum_{i \in K} \partial_{(\mathbf{w},b)} L^{(i)}(\mathbf{w},b)$$

Case Study: $f(x) = mx + b$



Gradient Descent (cont.)

- Steps of the algorithm:
  1. initialize the values of the model parameters, typically at random
  2. iteratively update the parameters in the direction of the negative gradient.

Notation:

θ: model parameters ($\mathbf{w}$,b)

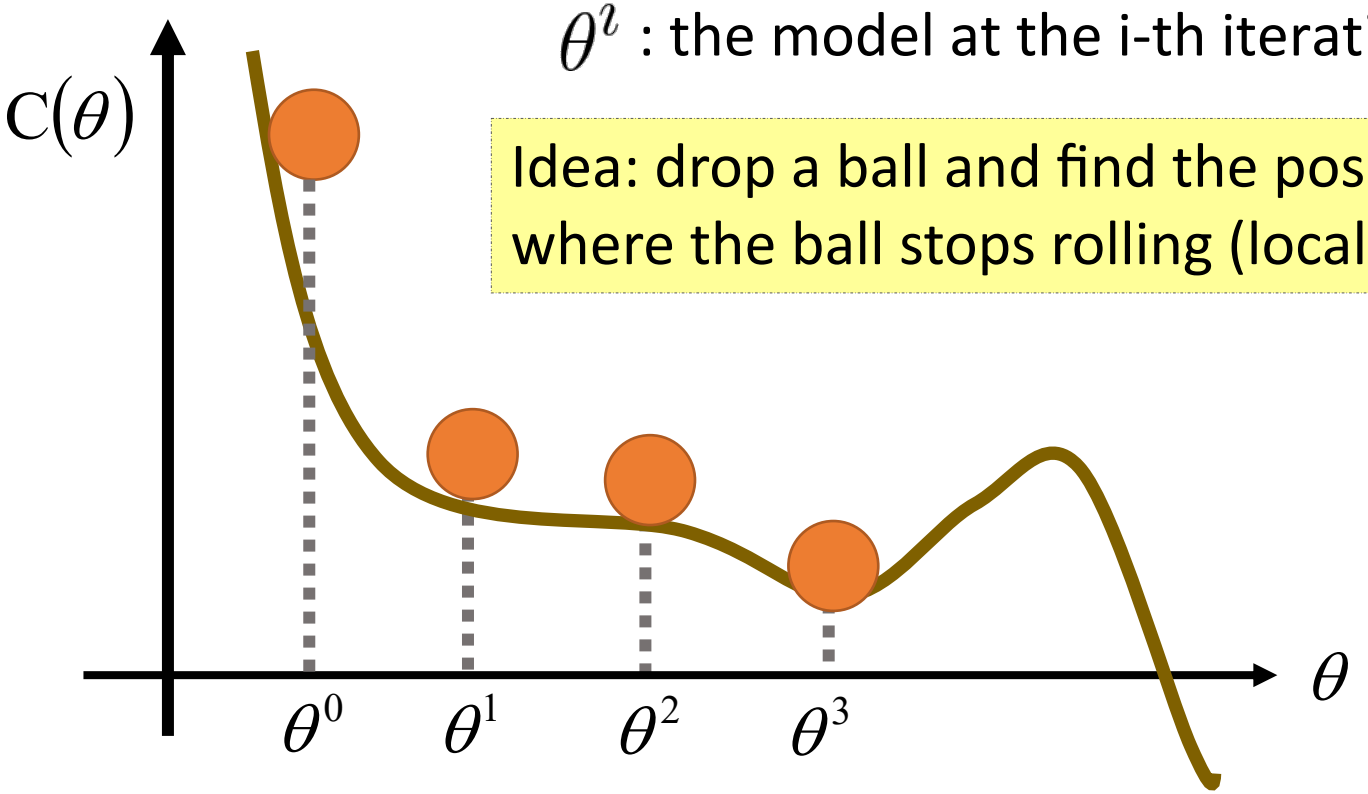
$\mathrm{C}(\theta)$: L ($\mathbf{w}$,b)

Gradient Descent (cont.)

- Assume that θ has only one variable

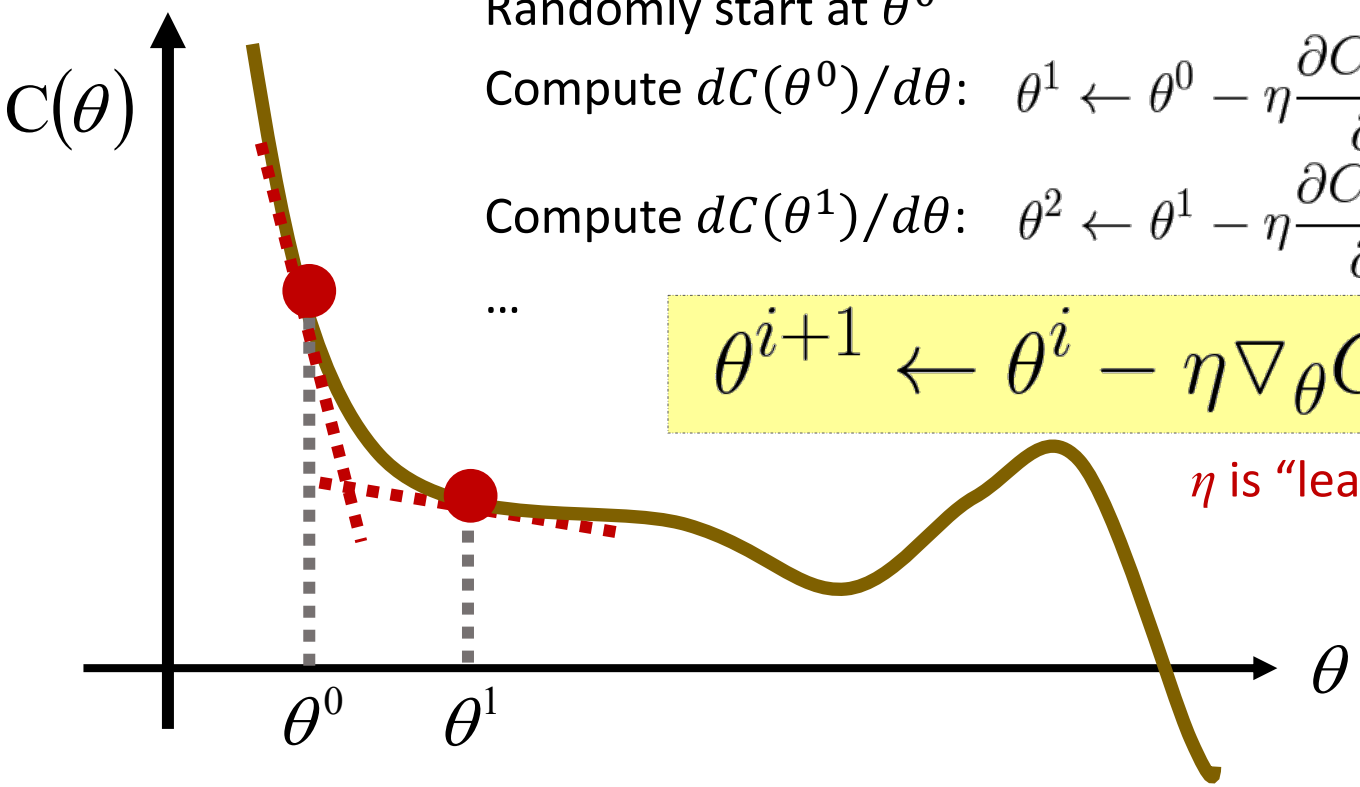
θ^i : the model at the i-th iteration

Idea: drop a ball and find the position where the ball stops rolling (local minima)



Gradient Descent (cont.)

- Assume that θ has only one variable



Randomly start at θ^0

Compute $dC(\theta^0)/d\theta$: $\theta^1 \leftarrow \theta^0 - \eta \frac{\partial C(\theta^0)}{\partial \theta}$

Compute $dC(\theta^1)/d\theta$: $\theta^2 \leftarrow \theta^1 - \eta \frac{\partial C(\theta^1)}{\partial \theta}$

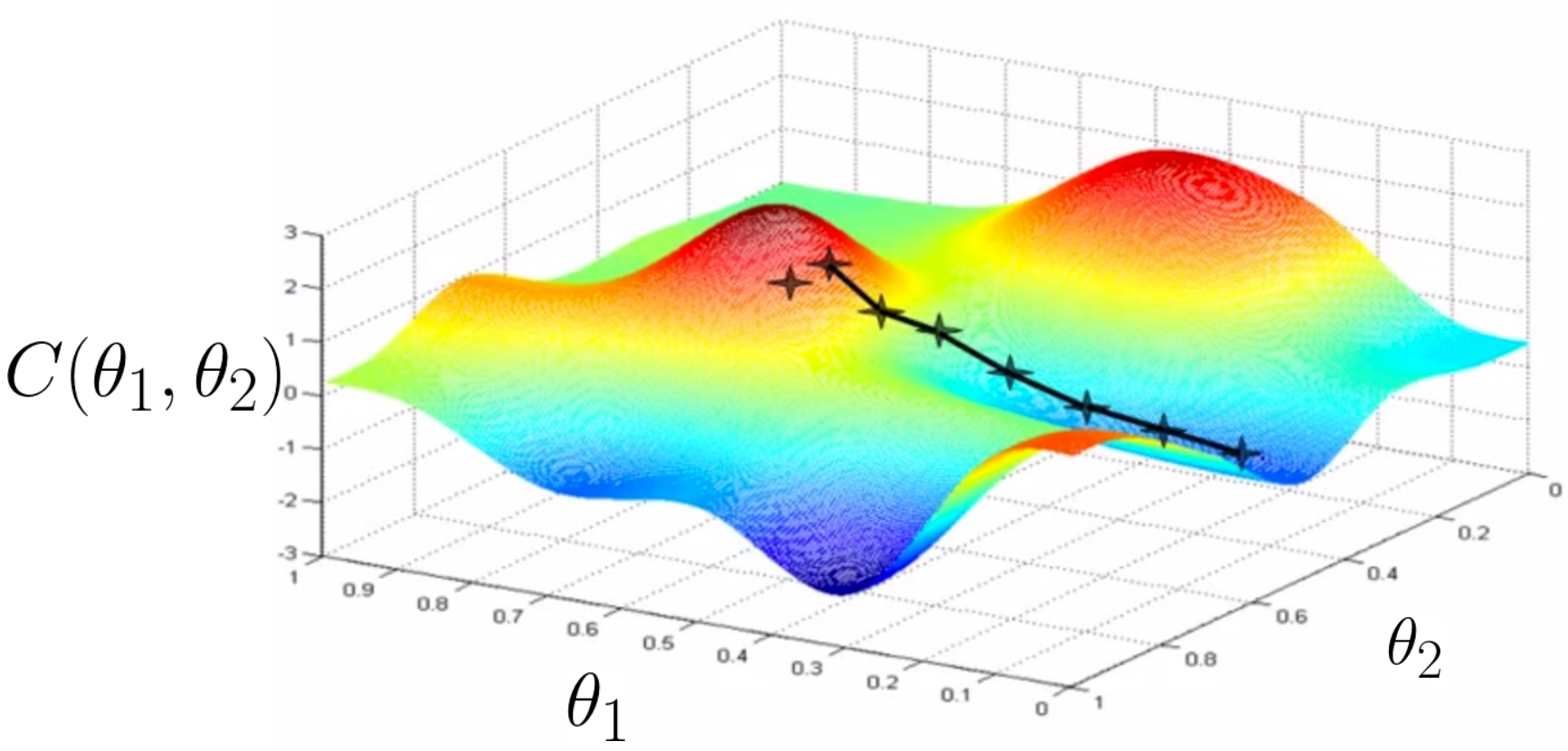
…

$$\theta^{i+1} \leftarrow \theta^i - \eta \nabla_\theta C(\theta^i)$$

η is "learning rate"

Gradient Descent (cont.)

- Assume that θ has two variables $\{\theta_1, \theta_2\}$



Gradient Descent (cont.)

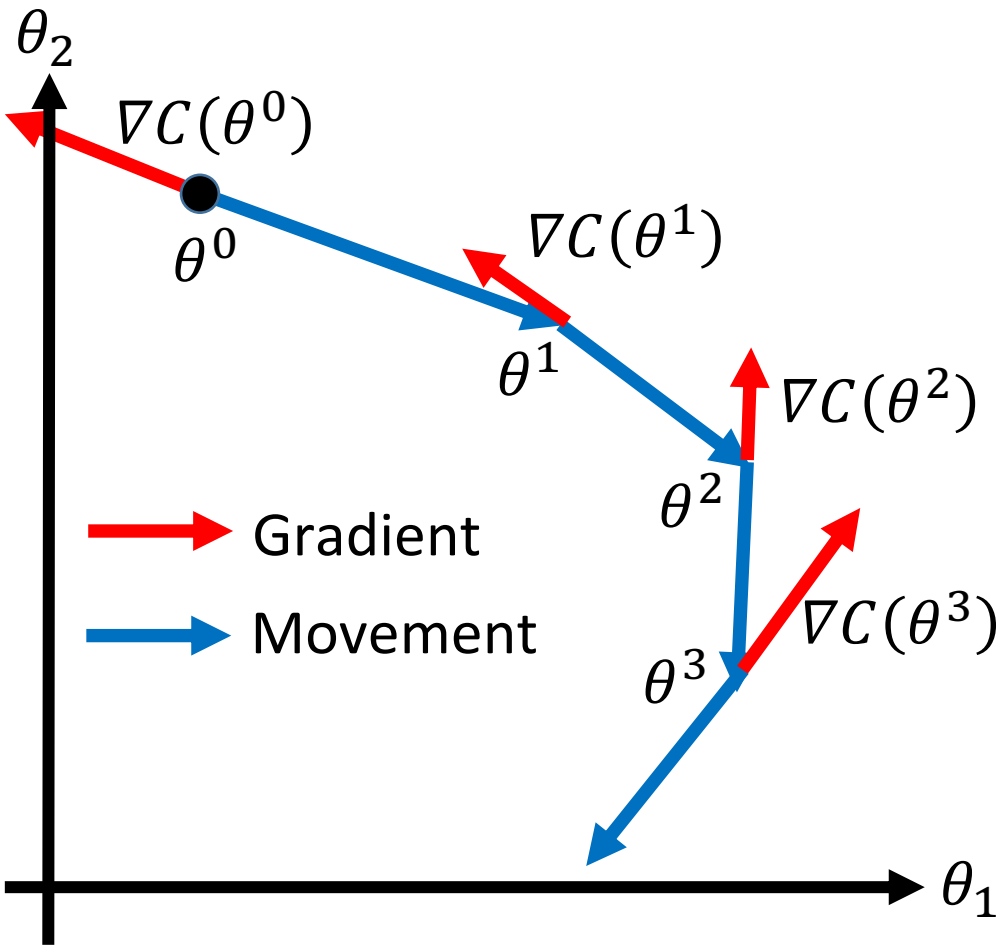
- Assume that θ has two variables $\{\theta_1, \theta_2\}$
  - Randomly start at θ^0: $\theta^0 = \begin{bmatrix} \theta_1^0 \\ \theta_2^0 \end{bmatrix}$
  - Compute the gradients of $C(\theta)$ at θ^0: $\nabla_\theta C(\theta^0) = \begin{bmatrix} \frac{\partial C(\theta_1^0)}{\partial \theta_1} \\ \frac{\partial C(\theta_2^0)}{\partial \theta_2} \end{bmatrix}$
  - Update parameters:

$$\begin{bmatrix} \theta_1^1 \\ \theta_2^1 \end{bmatrix} = \begin{bmatrix} \theta_1^0 \\ \theta_2^0 \end{bmatrix} - \eta \begin{bmatrix} \frac{\partial C(\theta_1^0)}{\partial \theta_1} \\ \frac{\partial C(\theta_2^0)}{\partial \theta_2} \end{bmatrix}$$

$$\theta^{i+1} \leftarrow \theta^i - \eta \nabla_\theta C(\theta^i)$$

  - Compute the gradients of $C(\theta)$ at θ^1: $\nabla_\theta C(\theta^1) = \begin{bmatrix} \frac{\partial C(\theta_1^1)}{\partial \theta_1} \\ \frac{\partial C(\theta_2^1)}{\partial \theta_2} \end{bmatrix}$

Gradient Descent (cont.)



Algorithm

Initialization: start at θ^0

while($\theta^{(i+1)} \neq \theta^i$)

{

compute gradient at θ^i

update parameters

$$\theta^{i+1} \leftarrow \theta^i - \eta \nabla_\theta C(\theta^i)$$

}

An Issue with Gradient Descent

$$\theta^{i+1} = \theta^{i} - \eta \nabla C(\theta^{i})$$

Training Data
$\{(x_1, \hat{y}_1), (x_2, \hat{y}_2), \ldots\}$

$$C(\theta) = \frac{1}{K} \sum_{k} \| f(x_k; \theta) - \hat{y}_k \| = \frac{1}{K} \sum_{k} C_k(\theta)$$

$$\nabla C(\theta^{i}) = \frac{1}{K} \sum_{k} \nabla C_k(\theta^{i})$$

After seeing all training samples, the model can be updated → slow

Stochastic Gradient Descent (SGD)

- Gradient Descent

$$\theta^{i+1} = \theta^i - \eta \nabla C(\theta^i) \quad \nabla C(\theta^i) = \boxed{\frac{1}{K}\sum_k \nabla C_k(\theta^i)}$$

- Stochastic Gradient Descent (SGD)
  - Pick a training sample x_k

$$\theta^{i+1} = \theta^i - \eta \nabla C_k(\theta^i)$$

Training Data
$\{(x_1, \hat{y}_1), (x_2, \hat{y}_2), \ldots\}$

  - If all training samples have the same probability to be picked, then

$$E[\nabla C_k(\theta^i)] = \boxed{\frac{1}{K}\sum_k \nabla C_k(\theta^i)}$$

The model can be updated after seeing one training sample → faster

Epoch (Definition)

- When running SGD, the model starts θ^0

Training Data
$\{(x_1, \hat{y}_1), (x_2, \hat{y}_2), \ldots\}$

pick x_1 $\quad \theta^1 = \theta^0 - \eta \nabla C_1(\theta^0)$

pick x_2 $\quad \theta^2 = \theta^1 - \eta \nabla C_2(\theta^1)$

$\vdots$

pick x_k $\quad \theta^k = \theta^{k-1} - \eta \nabla C_k(\theta^{k-1})$

$\vdots$

pick x_K $\quad \theta^K = \theta^{K-1} - \eta \nabla C_K(\theta^{K-1})$

see all training samples once

→ one epoch

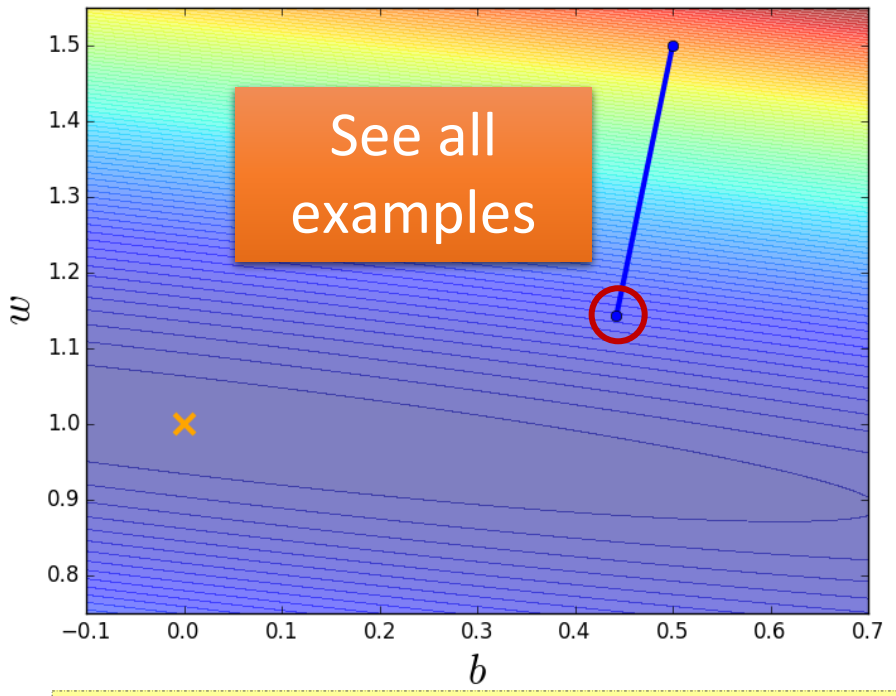
---

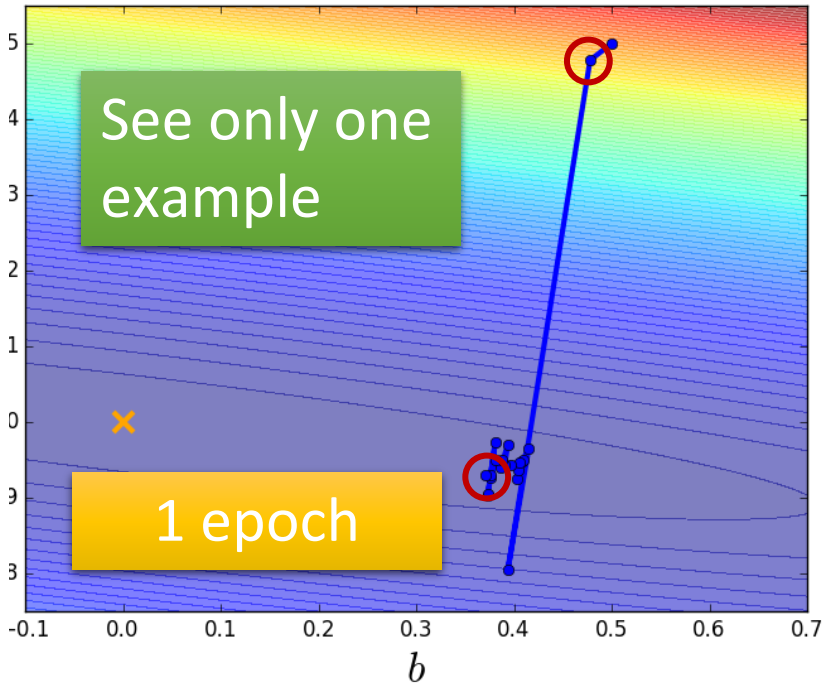
pick x_1 $\quad \theta^{K+1} = \theta^K - \eta \nabla C_1(\theta^K)$

Gradient Descent v.s. SGD

- Gradient Descent
- Update after seeing all the examples

- Stochastic Gradient Descent
- If there are 20 examples, update 20 times in one epoch.





SGD approaches the target faster than gradient descent

Mini-Batch SGD

- Batch Gradient Descent

$$\theta^{i+1} = \theta^i - \eta \frac{1}{K} \sum_k \nabla C_k(\theta^i)$$

Use all K samples in each iteration

- Stochastic Gradient Descent (SGD)
  - Pick a training sample x_k

$$\theta^{i+1} = \theta^i - \eta \nabla C_k(\theta^i)$$

Use 1 samples in each iteration

- Mini-Batch SGD
  - Pick a set of B training samples as a batch b

B is "batch size"

$$\theta^{i+1} = \theta^i - \eta \frac{1}{B} \sum_{x_k \in b} \nabla C_k(\theta^i)$$

Use all B samples in each iteration

Mini-Batch SGD

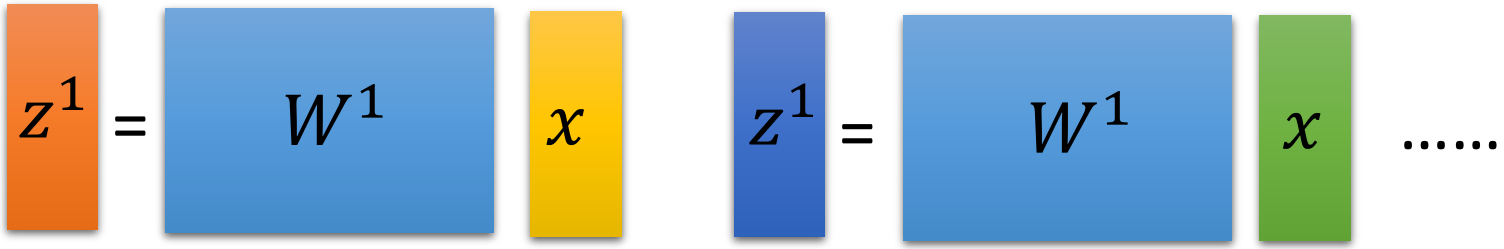
Algorithm 8.1 Stochastic gradient descent (SGD) update at training iteration k

Require: Learning rate ϵ_k.
Require: Initial parameter $\boldsymbol{\theta}$
  while stopping criterion not met **do**
    Sample a minibatch of m examples from the training set $\{\boldsymbol{x}^{(1)}, \ldots, \boldsymbol{x}^{(m)}\}$ with corresponding targets $\boldsymbol{y}^{(i)}$.
    Compute gradient estimate: $\hat{\boldsymbol{g}} \leftarrow +\frac{1}{m} \nabla_{\boldsymbol{\theta}} \sum_i L(f(\boldsymbol{x}^{(i)}; \boldsymbol{\theta}), \boldsymbol{y}^{(i)})$
    Apply update: $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \epsilon \hat{\boldsymbol{g}}$
  end while

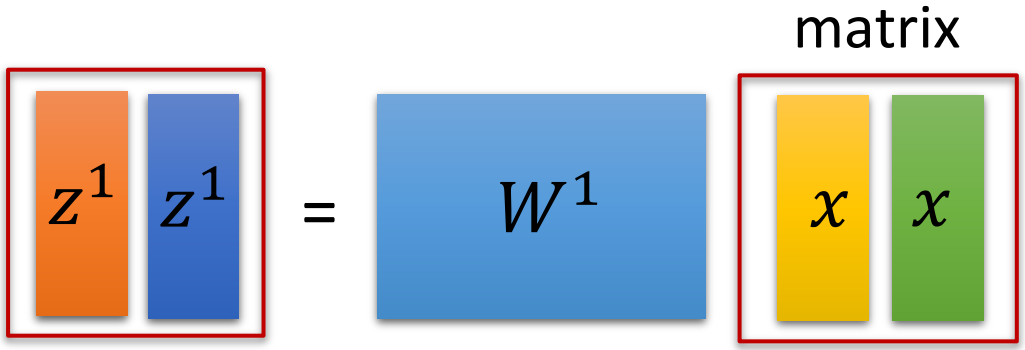
SGD vs. Mini-Batch

Training speed: mini-batch > SGD > Gradient Descent

- Stochastic Gradient Descent (SGD)



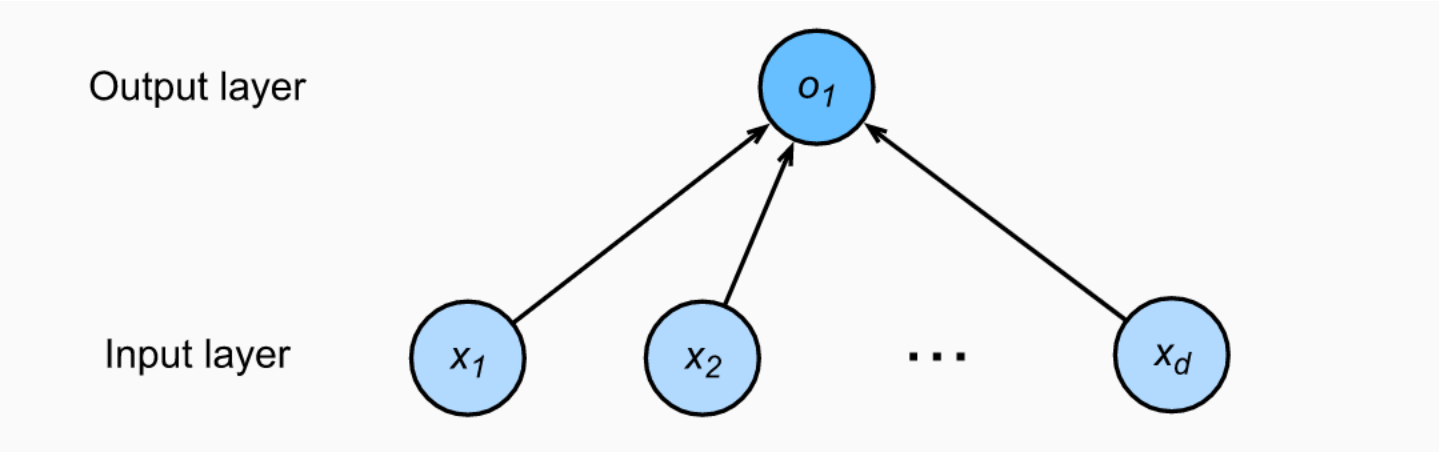
- Mini-Batch SGD



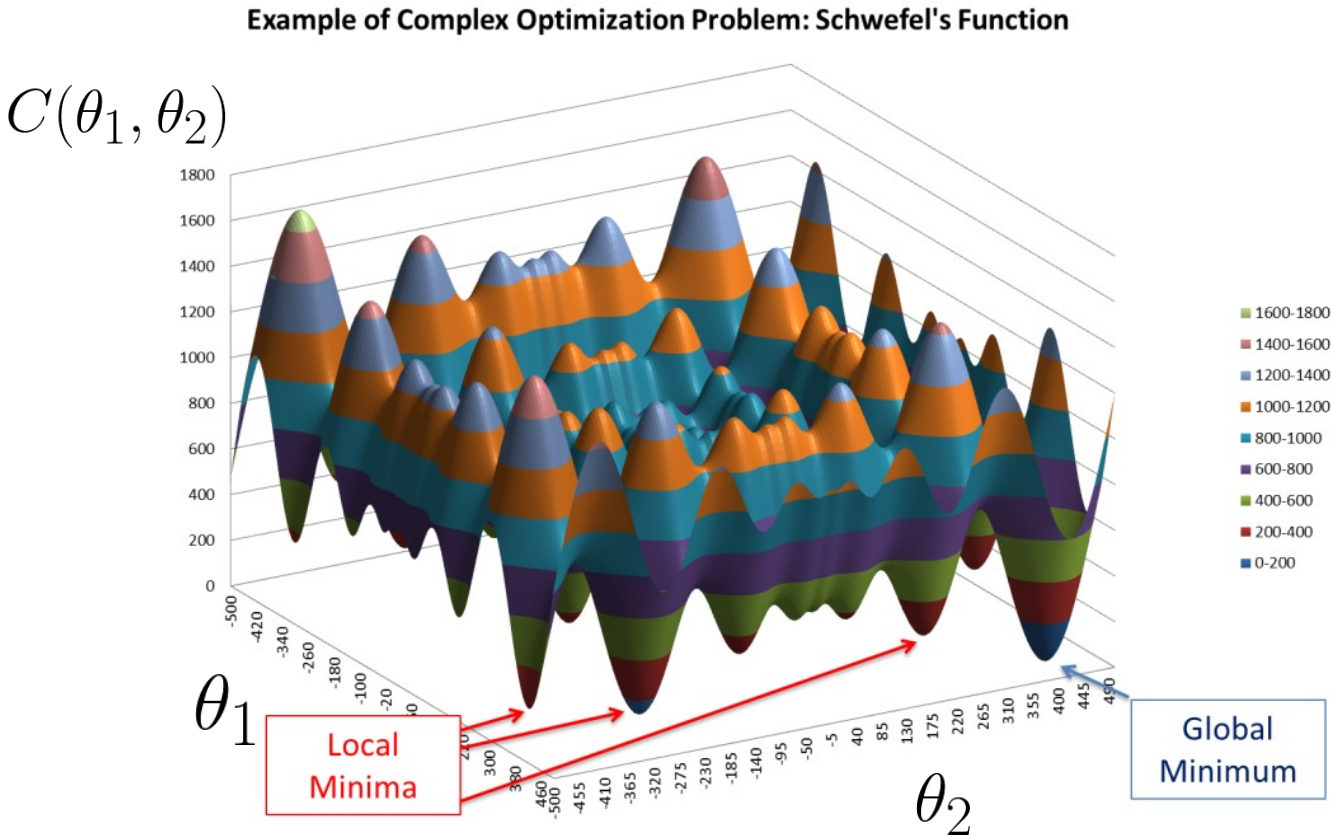
Modern computers run matrix-matrix multiplication faster than matrix-vector multiplication

From Linear Regression to Neural Networks

- Linear regression is a single-layer neural network, consisting of just a single neuron!
- Fully connected layer (also called *dense* layer): every input is connected to every output.



Big Issue: Local Optima



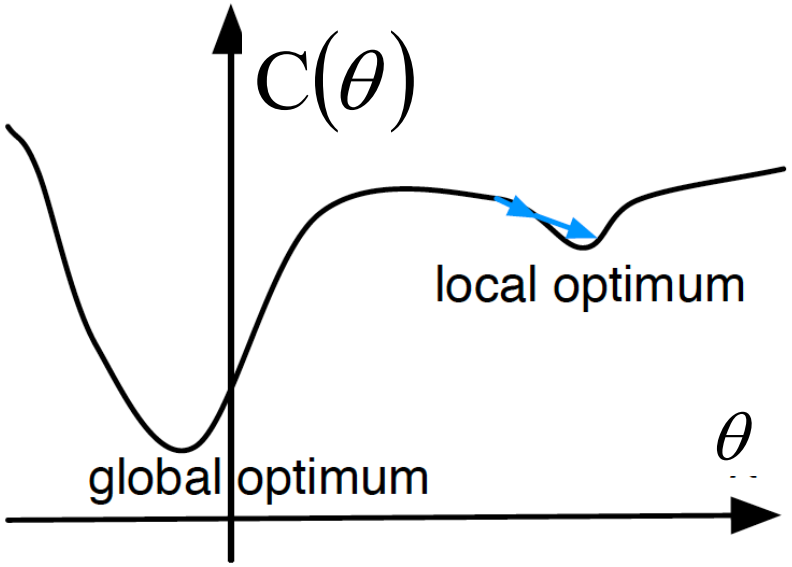
Neural networks have <u>no guarantee</u> for obtaining the global optimal solution!

Outline for Today

- Linear Regression
- Gradient Descent (and versions)
- Practical Tips
- Softmax Regression
- Activation and Loss Functions using PyTorch

Practical Tips: Initialization

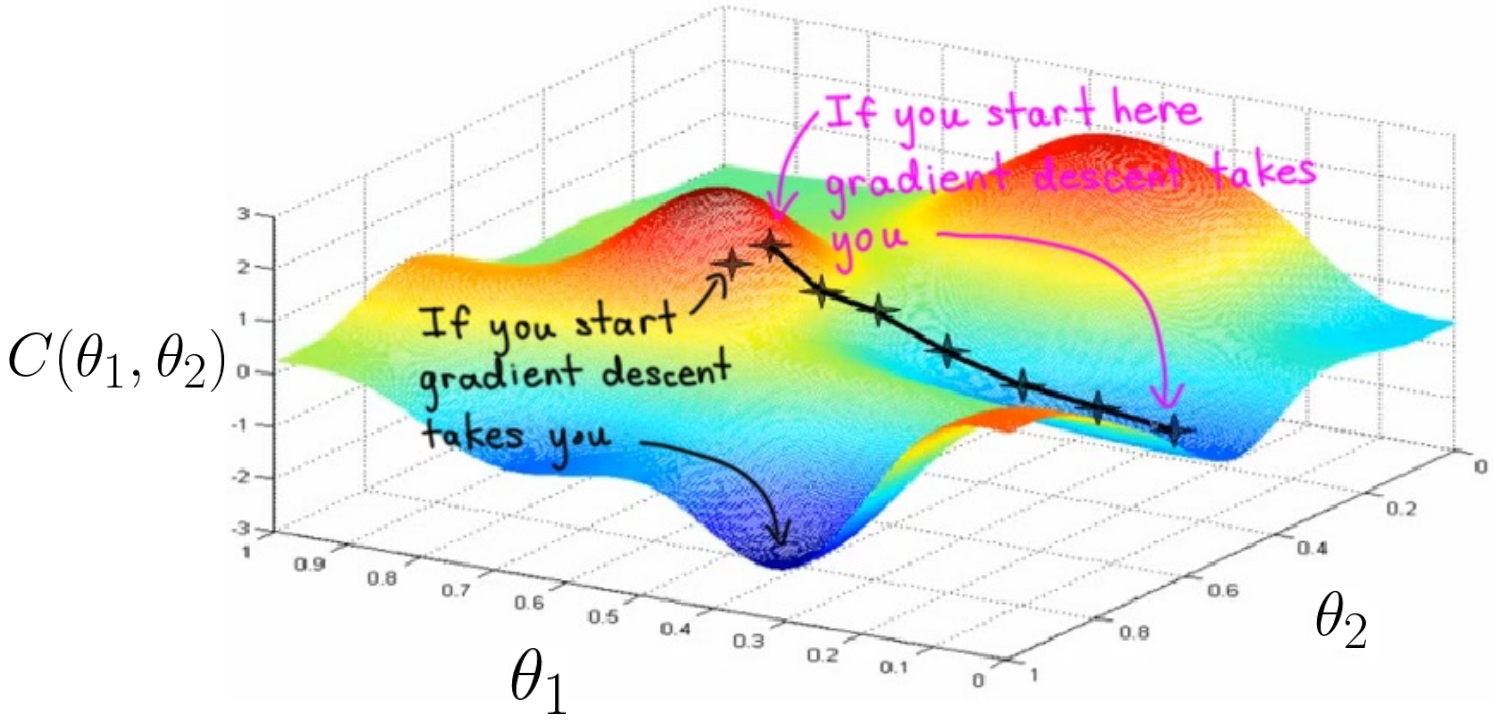
- Different initialization parameters may result in different trained models



Local optimum

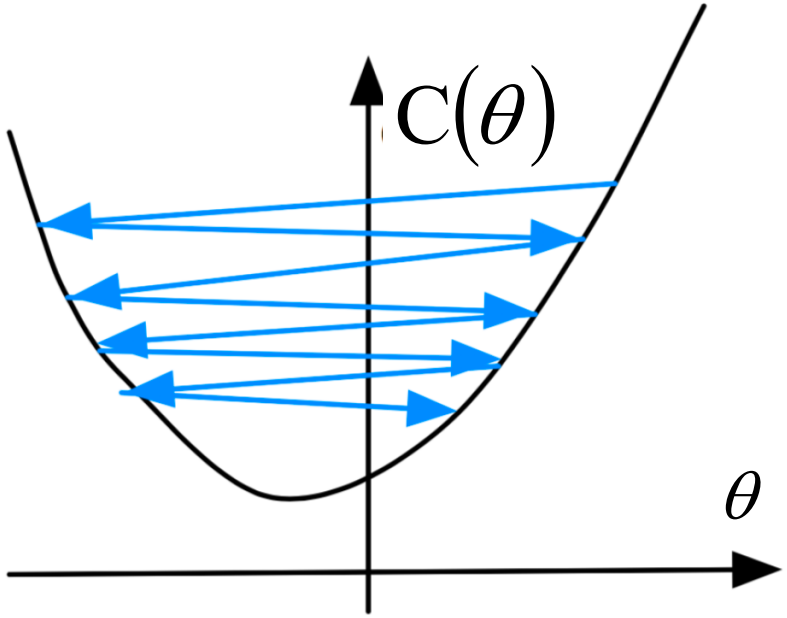
Practical Tips: Initialization

- Different initialization parameters may result in different trained models



Do not initialize the parameters equally → set them randomly

Practical Tips: Learning Rate



Too high learning rate

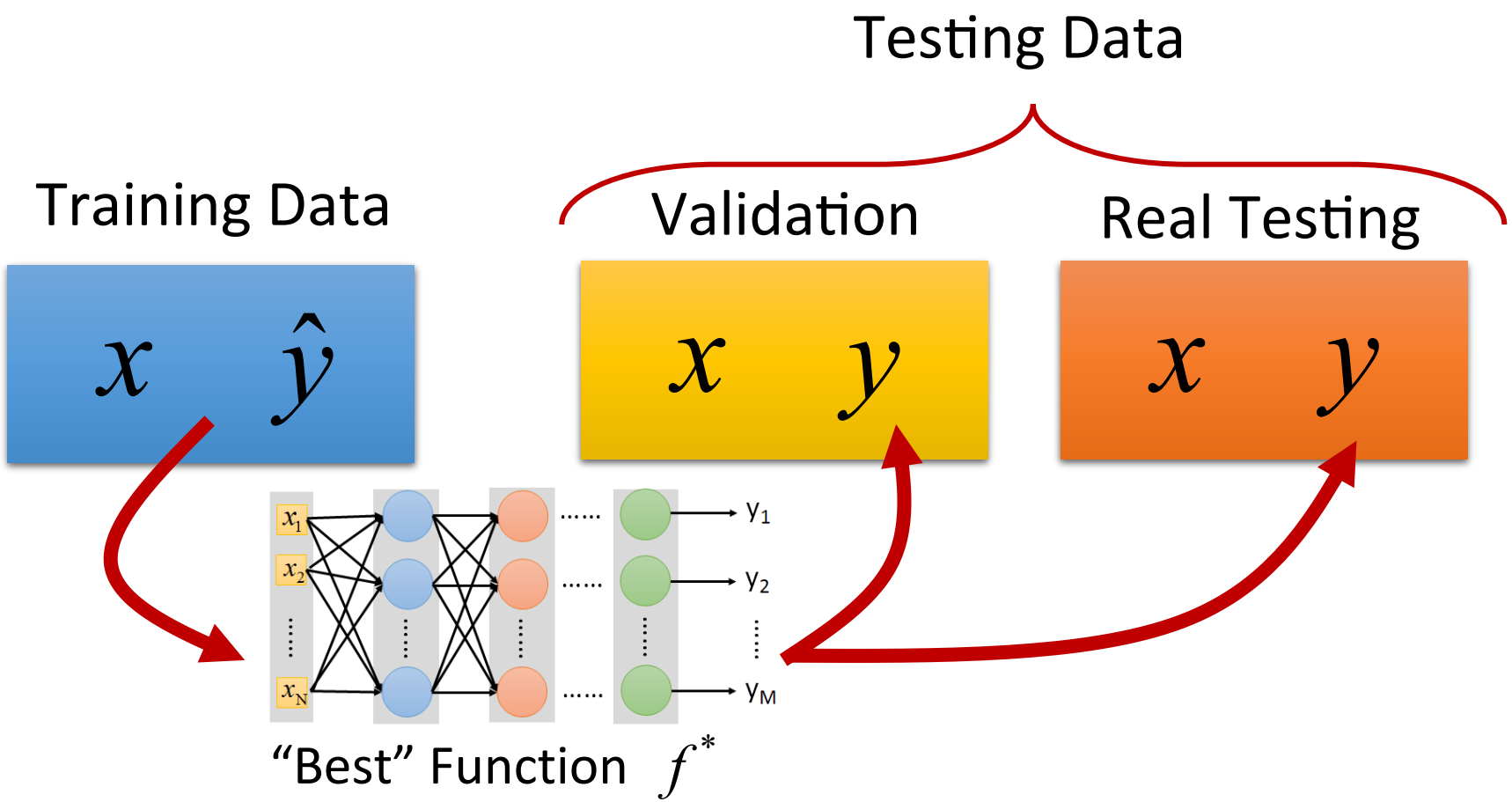
$$\theta^{i+1} \leftarrow \theta^{i} - \eta \nabla_{\theta} C(\theta^{i})$$

Learning rate should be set carefully

Tips for Mini-Batch Training

- Shuffle training samples before every epoch
  - The network might memorize the order you feed the examples
- Use a fixed batch size for every epoch
  - Enable fast implementation of matrix multiplication for calculations
- Adapt the learning rate to the batch size
  - Larger batch → smaller learning rate

Learning Recipe



Learning Recipe

Testing Data

Training Data

$x \quad \hat{y}$

Validation

$x \quad y$

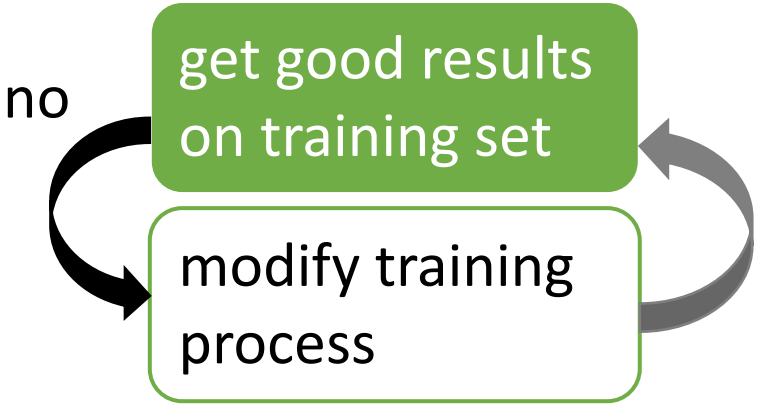
immediately observe the performance

Real Testing

$x \quad y$

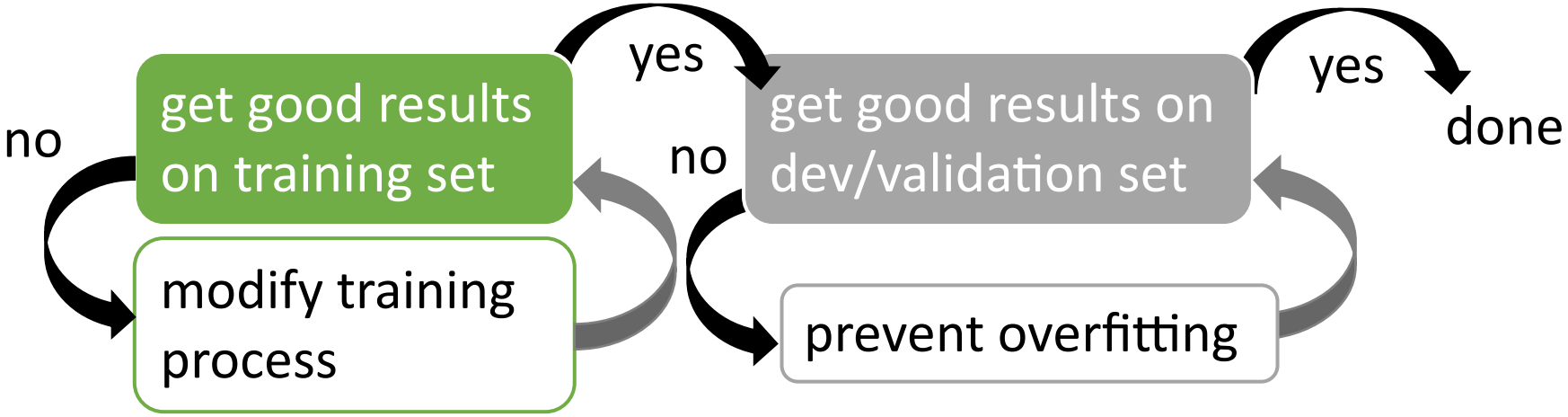
Will not know the performance immediately

Learning Recipe



- Possible reasons
  - No good function exists: bad hypothesis function set
    → **Reconstruct the model architecture**
  - Cannot find a good function: local optima
    → **Change the training strategy**

Learning Recipe



Better performance on training but worse performance on dev → overfitting

Overfitting

- Possible solutions
  - More training samples
  - Dropout, etc.

Outline for Today

- Linear Regression
- Gradient Descent (and versions)
- Practical Tips
- Softmax Regression
- Activation and Loss Functions using PyTorch

Softmax Regression

- Regression is useful for “how much?” and “how many?” questions, where the target can take a real value.
- Many NLP problems are about “which one?”
  - Sentiment classification: positive, negative, neutral
  - Sentence boundary detection from text: is the punctuation mark defining a sentence boundary or not
  - What is the part-of-speech tag of the word “word” in this sentence? NOUN, VERB, etc.?
- We can represent target labels with one-hot encodings as well. For example, for sentiment classification:

$$y \in \{\underbrace{(1,0,0)}_{\text{Positive}}, \underbrace{(0,1,0)}_{\text{Negative}}, \underbrace{(0,0,1)}_{\text{Neutral}}\}.$$

Network Architecture for Multi-Class Problems

- To estimate the conditional probabilities associated with each class, we need a model with multiple outputs, one per class.
- As many linear functions as we have outputs.
- Example: Assume our input is represented with 4 features, then to compute the logits o_1, o_2, o_3, for each output, we need:

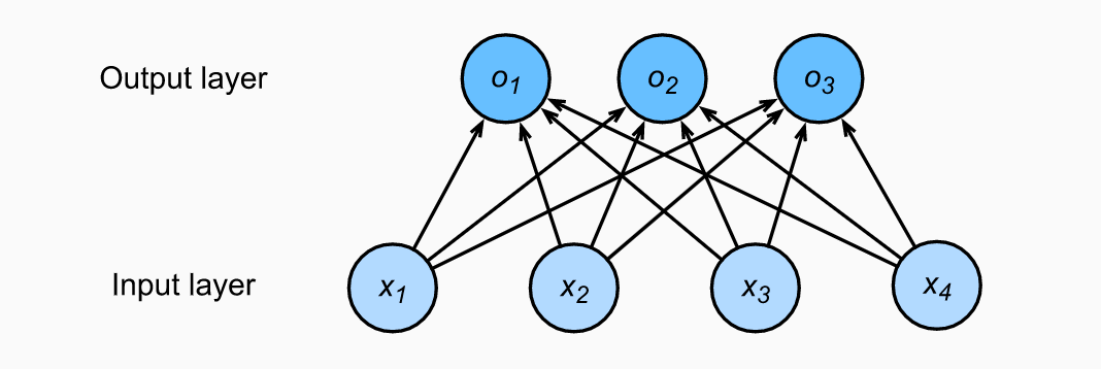
$$o_1 = x_1 w_{11} + x_2 w_{12} + x_3 w_{13} + x_4 w_{14} + b_1$$

$$o_2 = x_1 w_{21} + x_2 w_{22} + x_3 w_{23} + x_4 w_{24} + b_2$$

$$o_3 = x_1 w_{31} + x_2 w_{32} + x_3 w_{33} + x_4 w_{34} + b_3$$

$$\mathbf{o} = \mathbf{W}\mathbf{x} + \mathbf{b}$$

3 x 4 matrix



Multi-Class Problems

- Interpret model outputs as probabilities and optimize our parameters to produce probabilities that maximize the likelihood of the observed data
- To generate predictions, we can set a threshold or choose the *argmax* of the predicted probabilities

- Interpret the logits o directly as our outputs of interest, however:
  - Nothing constrains these numbers to sum to 1.
  - Depending on the inputs, they can take negative values.

Softmax

- Transforms logits such that they become nonnegative and sum to 1, while requiring that the model remains differentiable.

$$\hat{\mathbf{y}} = \mathrm{softmax}(\mathbf{o}) \quad \text{where} \quad \hat{y}_i = \frac{\exp(o_i)}{\sum_j \exp(o_j)}$$

- $\hat{y}_1+\hat{y}_2+\hat{y}_3 = 1$,
- $0 \leq \hat{y}_i \leq 1$ for all i, and
- The ordering of the logits has not changed, hence we can still pick the output using argmax!

Maximum Likelihood Estimation

- The softmax function gives us a vector $\hat{\mathbf{y}}$, interpreted as estimated conditional probabilities of each class given the input x, e.g., $\hat{y}_1 = P$ (y=cat|$\mathbf{x}$).

$$P(Y \mid X) = \prod_{i=1}^{n} P(y^{(i)} \mid x^{(i)})$$

$$-\log P(Y \mid X) = \sum_{i=1}^{n} -\log P(y^{(i)} \mid x^{(i)})$$

- Maximizing $P(Y|X)$ (and thus equivalently minimizing $-\log P(Y|X)$) corresponds to predicting the label well!

Cross-Entropy Loss and Softmax

$$l(\mathbf{y}, \hat{\mathbf{y}}) = -\sum_{j=1}^{q} y_j \log \hat{y}_j.$$

q: the number of classes

- If we add o into the definition of the loss l and use the definition of the softmax we obtain:

$$\begin{aligned} l(\mathbf{y}, \hat{\mathbf{y}}) &= -\sum_{j=1}^{q} y_j \log \frac{\exp(o_j)}{\sum_{k=1}^{q} \exp(o_k)} \\ &= \sum_{j=1}^{q} y_j \log \sum_{k=1}^{q} \exp(o_k) - \sum_{j=1}^{q} y_j o_j \\ &= \log \sum_{k=1}^{q} \exp(o_k) - \sum_{j=1}^{q} y_j o_j. \end{aligned}$$

- If we compute its derivative with respect to o_j, we get:

$$\partial_{o_j} l(\mathbf{y}, \hat{\mathbf{y}}) = \frac{\exp(o_j)}{\sum_{k=1}^{q} \exp(o_k)} - y_j = \mathrm{softmax}(\mathbf{o})_j - y_j.$$

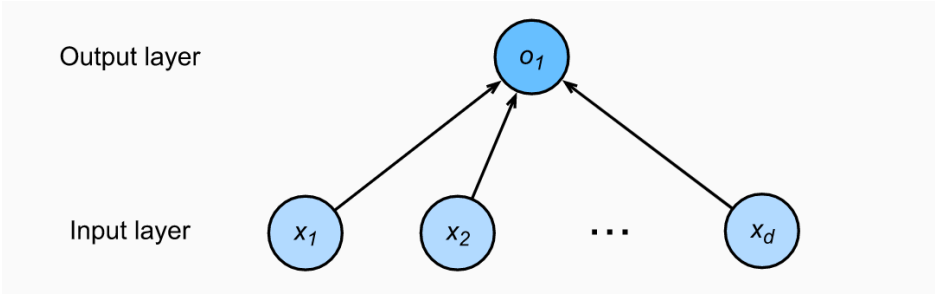
The gradient is the difference between the probability assigned to the true class by our model. This makes computing gradients very easy in practice!

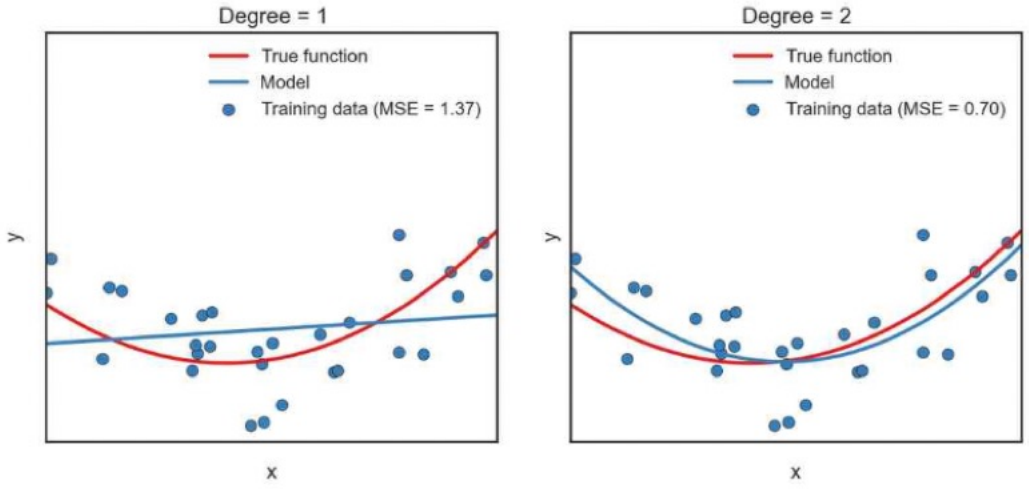
Outline for Today

- Linear Regression
- Gradient Descent (and versions)
- Practical Tips
- Softmax Regression
- Activation and Loss Functions using PyTorch

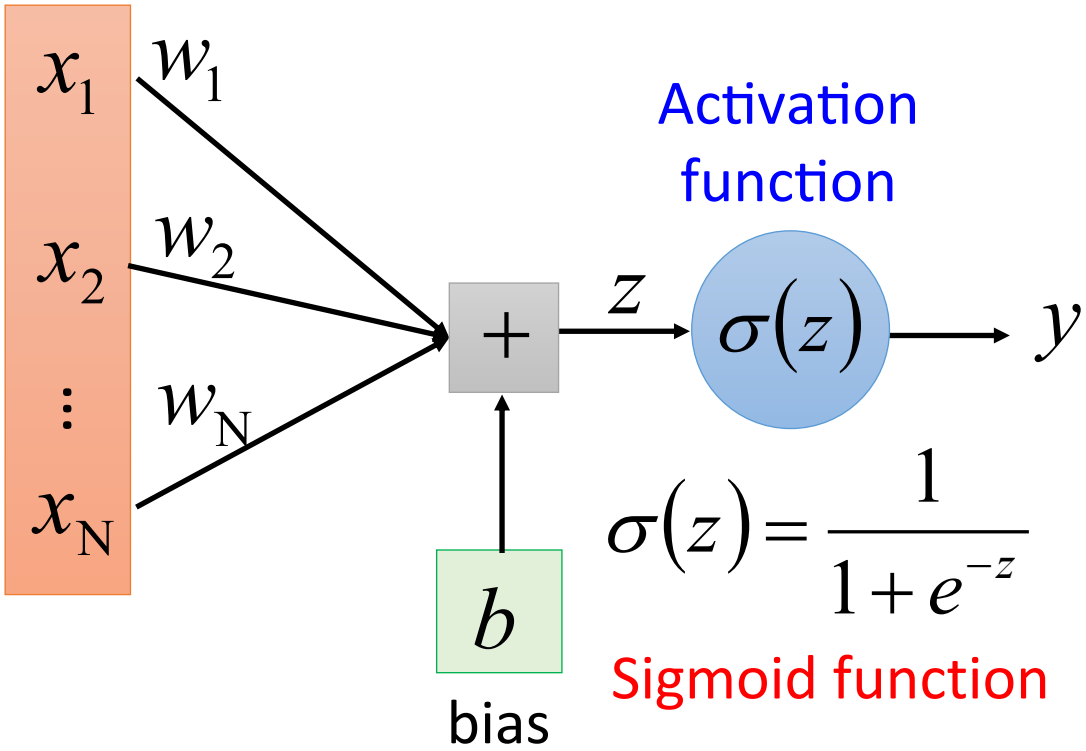
Non-linearity

- Linear regression inputs directly to our outputs via a single linear transformation.
- But what if our data is not linear?



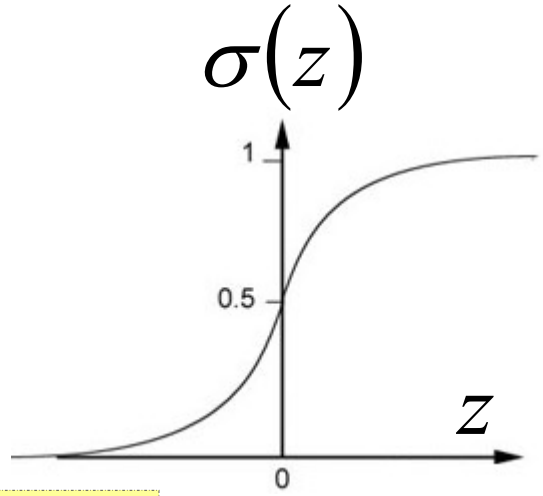


Perceptron – A single neuron



$$\sigma(z) = \frac{1}{1+e^{-z}}$$

Sigmoid function



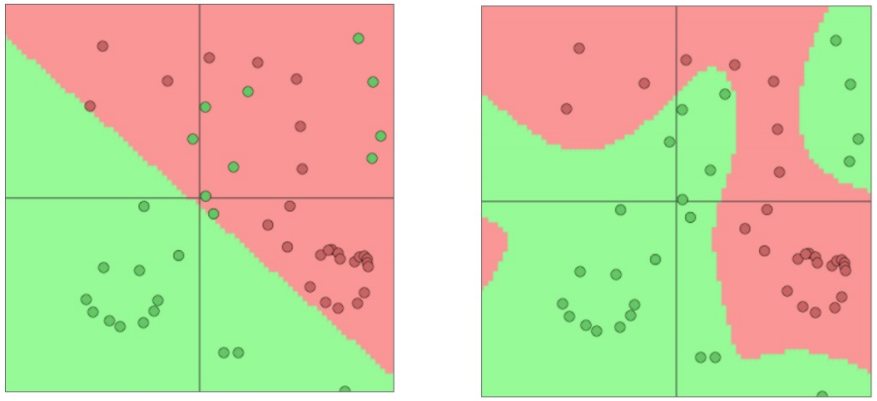
Each neuron is a very simple function

Why Non-Linearity?

- Function approximation
  - ***Without non-linearity***, deep neural networks work the same as linear transform

$$W_1(W_2 \cdot x) = (W_1 W_2)x = Wx$$

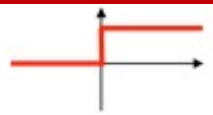
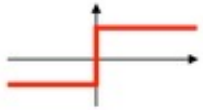
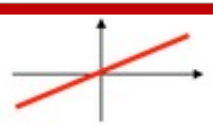
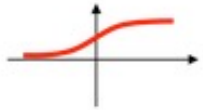
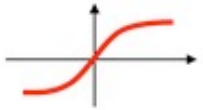
  - ***With non-linearity***, networks with more layers can approximate more complex functions



Activation Functions

$$\sigma(\cdot)$$

Non-linear functions are frequently used in neural nets

Activation function	Equation	Example	1D Graph
Unit step (Heaviside)	$\phi(z) = \begin{cases} 0, & z < 0, \\ 0.5, & z = 0, \\ 1, & z > 0, \end{cases}$	Perceptron variant	
Sign (Signum)	$\phi(z) = \begin{cases} -1, & z < 0, \\ 0, & z = 0, \\ 1, & z > 0, \end{cases}$	Perceptron variant	
Linear	$\phi(z) = z$	Adaline, linear regression	
Piece-wise linear	$\phi(z) = \begin{cases} 1, & z \geq \frac{1}{2}, \\ z + \frac{1}{2}, & -\frac{1}{2} < z < \frac{1}{2}, \\ 0, & z \leq -\frac{1}{2}, \end{cases}$	Support vector machine	
Logistic (sigmoid)	$\phi(z) = \frac{1}{1 + e^{-z}}$	Logistic regression, Multi-layer NN	
Hyperbolic tangent	$\phi(z) = \frac{e^{z} - e^{-z}}{e^{z} + e^{-z}}$	Multi-layer NN	

boolean

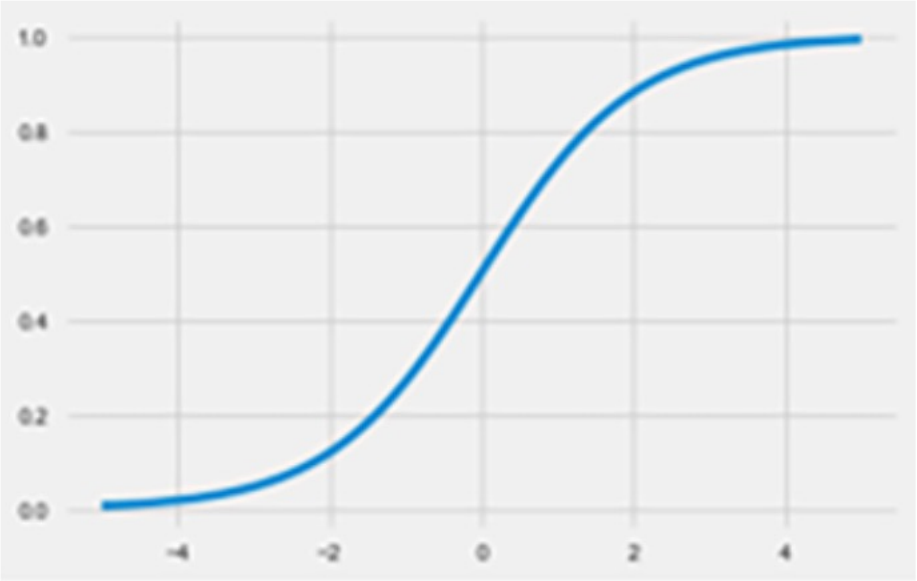
linear

non-linear

Activation Functions in PyTorch - Sigmoid

```
import torch
import matplotlib.pyplot as plt

x = torch.range(-5., 5., 0.1)
y = torch.sigmoid(x)
plt.plot(x.numpy(), y.numpy())
plt.show()
```

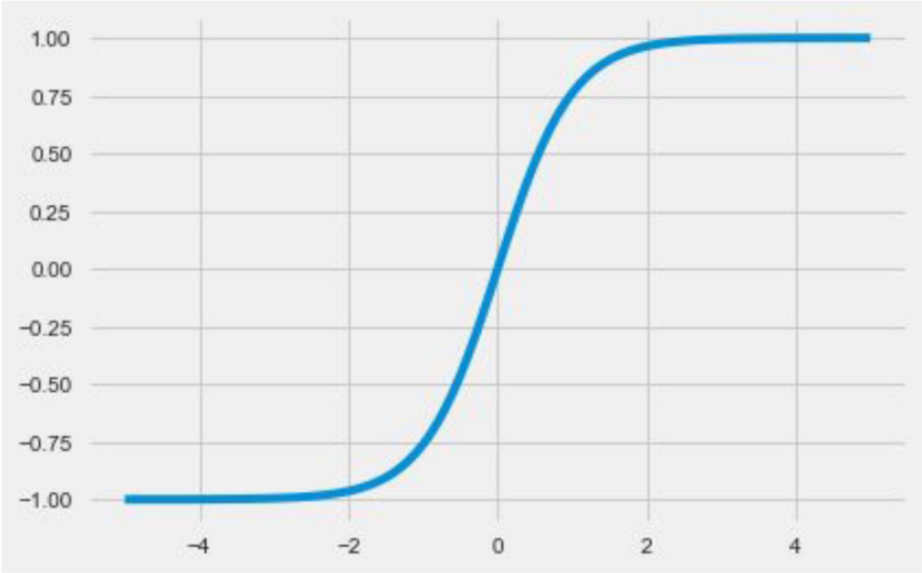


$$\text{sigmoid}(x) = \frac{1}{1 + e^{-x}}$$

Activation Functions in PyTorch- Tanh

```
import torch
import matplotlib.pyplot
as plt

x = torch.range(-5., 5.,
0.1)
y = torch.tanh(x)
plt.plot(x.numpy(),
y.numpy())
plt.show()
```



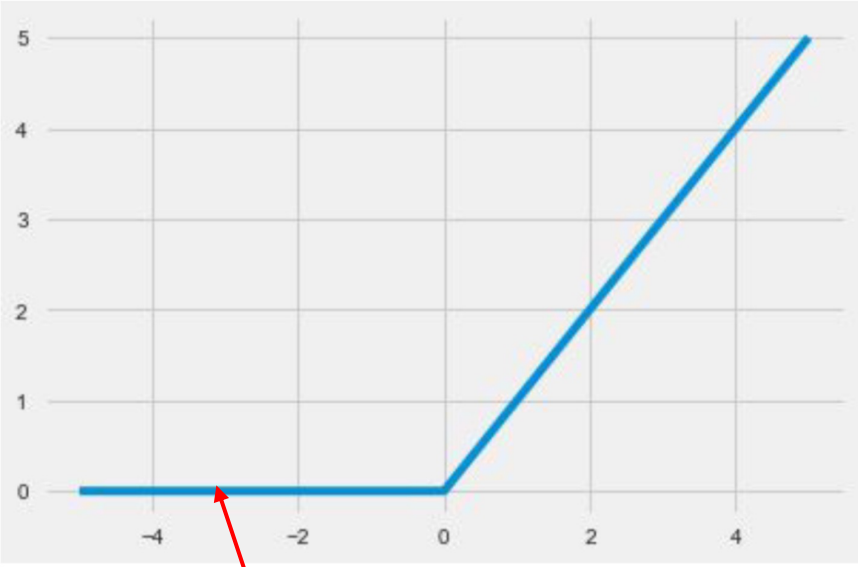
$$\tanh(x) = \frac{\sinh(x)}{\cosh(x)} = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

Activation Functions in PyTorch - ReLU

```
import torch
import matplotlib.pyplot
as plt

relu = torch.nn.ReLU()
x = torch.range(-5., 5.,
0.1)
y = relu(x)

plt.plot(x.numpy(),
y.numpy())
plt.show()
```



Clipping negative values to zero!

$$\mathrm{ReLU}(x) = \max(x, 0)$$

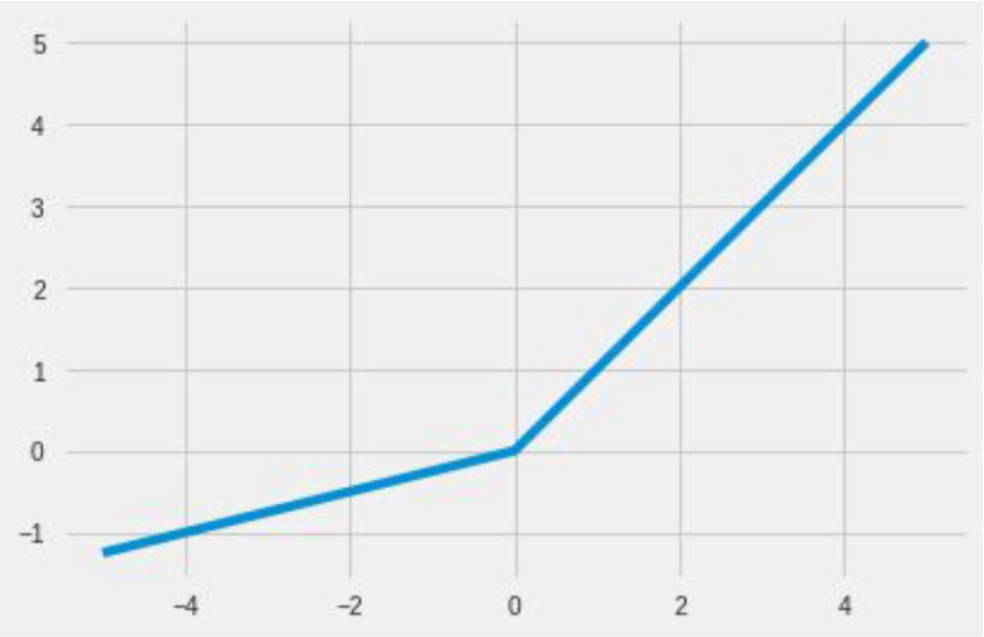
However, over time certain outputs in the network can simply become zero and never revive again.

Activation Functions in PyTorch- PReLU

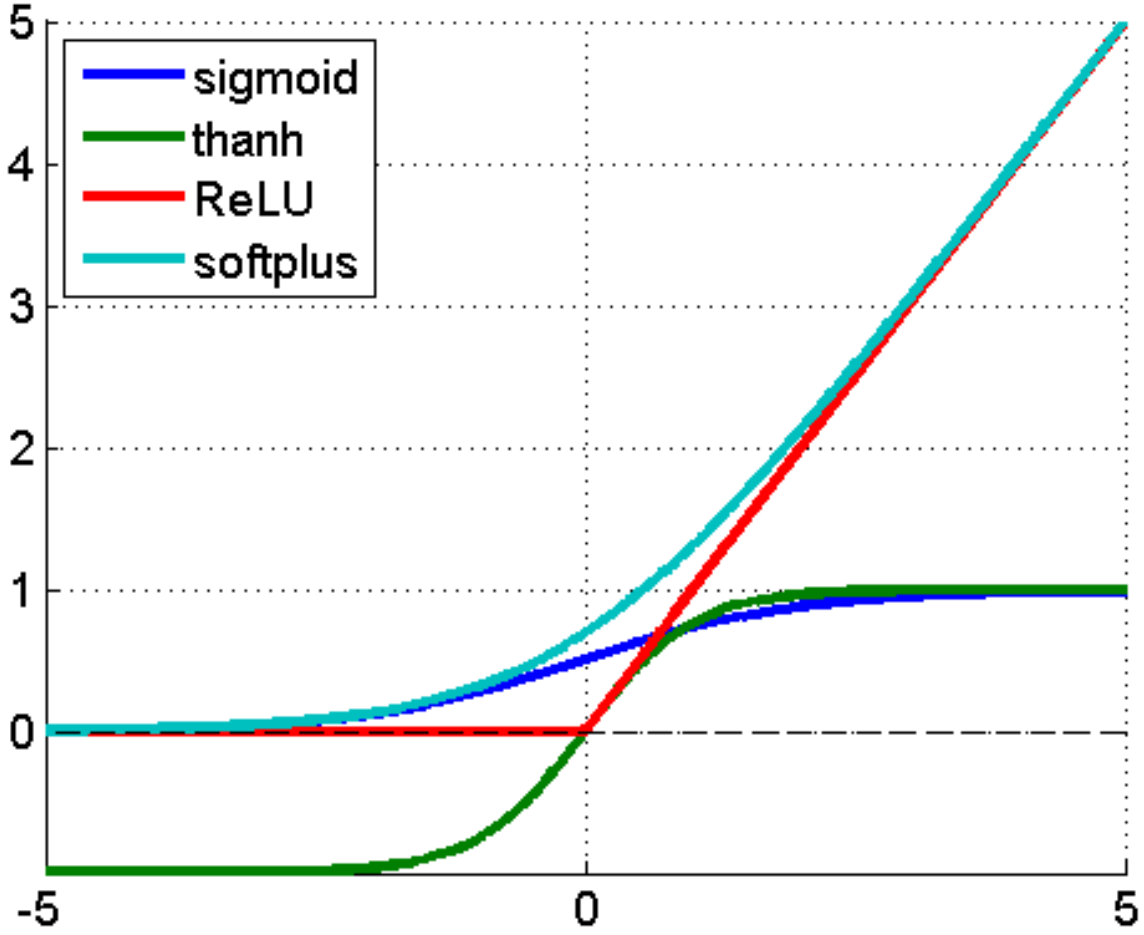
```
import torch
import matplotlib.pyplot as plt

prelu =
torch.nn.PReLU(num_parameters=1)
x = torch.range(-5., 5., 0.1)
y = prelu(x)

plt.plot(x.numpy(), y.numpy())
plt.show()
```



Non-Linear Activation Functions- Summary



Activation Functions in PyTorch - Softmax

```python
import torch
import torch.nn as nn

softmax = nn.Softmax(dim=1)
x_input = torch.randn(1,4)
y_output = softmax(x_input)
print(x_input)
print(y_output)
print(torch.sum(y_output, dim=1))
```

```
tensor([[ 0.4453, -0.8326, -0.4668,  0.5607]])
tensor([[0.3568, 0.0994, 0.1433, 0.4005]])
tensor([1.0000])
```

Frequently Used Loss Functions

- Square loss

$$C(\theta) = (1 - \hat{y} f(x;\theta))^2$$

- Hinge loss

$$C(\theta) = \max(0, 1 - \hat{y} f(x;\theta))$$

- Logistic loss

$$C(\theta) = -\hat{y} \log(f(x;\theta))$$

- Cross entropy loss

$$C(\theta) = -\sum \hat{y} \log(f(x;\theta))$$

- Others: large margin, etc.

$\hat{y}$: ground truth label
f(x; θ): estimation with model parameters θ.

https://en.wikipedia.org/wiki/Loss_functions_for_classification

Loss Functions in PyTorch – Mean Squared Error Loss

```python
import torch
import torch.nn as nn

mse_loss = nn.MSELoss()
outputs = torch.randn(3, 5, requires_grad=True)
targets = torch.randn(3, 5)
loss = mse_loss(outputs, targets)
print(outputs)
print(targets)
print(loss)
```

```
tensor([[ 0.3472, -0.2246,  0.1670, -0.2578,  1.5375],
        [ 0.1875, -0.9442,  0.2554,  1.4726,  1.3097],
        [ 1.6956, -0.2329,  0.7398,  0.6420,  2.1655]], requires_grad=True)
tensor([[ 4.4099e-01,  3.5975e-01, -3.3145e-02,  1.6507e+00, -9.6259e-02],
        [-1.3071e+00,  2.6525e-04, -3.9299e-01,  1.3937e+00,  1.0394e+00],
        [-9.4118e-02,  7.5348e-01, -2.3449e-01, -9.0545e-01, -1.3017e+00]])
tensor(1.9913, grad_fn=<MseLossBackward>)
```

Loss Functions in PyTorch-Categorical Cross-Entropy Loss

```python
import torch
import torch.nn as nn

ce_loss = nn.CrossEntropyLoss()
outputs = torch.randn(3, 5, requires_grad=True)
targets = torch.tensor([1, 0, 3], dtype=torch.int64)
loss = ce_loss(outputs, targets)
print(outputs)
print(targets)
print(loss)
```

```
tensor([[ 1.2979,  0.5740, -1.5945, -2.4253,  0.1673],
        [ 1.0170,  0.3193,  0.4528,  1.3819, -1.2214],
        [-0.3760,  0.6830,  0.5004, -1.6647,  0.2250]], requires_grad=True)
tensor([1, 0, 3])
tensor(2.0196, grad_fn=<NllLossBackward>)
```

- typically used in a multiclass classification, where the outputs are interpreted as predictions of class membership probabilities
- Nn.BCELoss() for the binary